FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Elijah Paul M. Nonan
Section:	AN-43
Professor:	Ms. Crisola Tan
Date:	11/04/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

Source code:

```
"""Technical Assessment 4: knowledge representation, rule-based reasoning, and CBR.

Use case: a small home energy-use advisor. This educational example is not intended to control
real appliances or replace professional advice.
"""

# ---------------------------------------------------------------------------
# Part 1: Knowledge Representation (KR)
# ---------------------------------------------------------------------------
```

```
# The five top-level entries are facts; each fact contains three categories.
# RBR evaluates these preset values. CBR separately asks about a new problem.
KNOWLEDGE_BASE = {
    "household": {
        "residents": 3,
        "home_during_day": False,
        "daytime_occupants": 0,
    },
    "energy": {
        "monthly_kwh": 420,
        "monthly_budget": 3500,
        "rate_per_kwh": 12,
    },
    "appliances": {
        "aircon_hours_daily": 9,
        "has_inverter_aircon": False,
        "aircon_temperature_c": 24,
    },
    "environment": {
        "weather": "hot",
        "outdoor_temperature_c": 32,
        "humidity_percent": 80,
    },
    "usage_patterns": {
        "peak_usage_period": "afternoon",
        "uses_fan": False,
        "switches_off_lights_when_away": True,
    },
}


# ------------------------------------------------------------------------
# Part 2: Rule-Based Reasoning (RBR)
# ------------------------------------------------------------------------

def rule_check_energy_use(facts):
    """Recommend checking high usage against a simple household benchmark."""
    usage = facts["energy"]["monthly_kwh"]
    residents = max(facts["household"]["residents"], 1)
    kwh_per_person = usage / residents
    if kwh_per_person > 100:
        return (f"Review monthly electricity use: {kwh_per_person:.0f} kWh per "
                "resident is above the 100 kWh guideline.")
    return None


def rule_reduce_aircon_hours(facts):
    """Suggest a modest adjustment when daily air-conditioner use is high."""
    hours = facts["appliances"]["aircon_hours_daily"]
    environment = facts["environment"]
    if (hours >= 8 and environment["weather"] == "hot"
            and environment["outdoor_temperature_c"] >= 30
            and environment["humidity_percent"] >= 70
            and facts["appliances"]["aircon_temperature_c"] < 26):
        return ("Try raising the air-conditioner temperature by 1–2°C and "
                "using a fan to reduce long cooling periods.")
    return None


def rule_use_timer(facts):
    """Suggest a timer when the home is empty during the day."""
    household = facts["household"]
    if not household["home_during_day"] and household["daytime_occupants"] == 0:
        return "Use an air-conditioner timer so cooling does not continue in an empty home."
    return None
```

```
def rule_check_budget(facts):
    """Flag an estimated bill above the household's stated budget."""
    # Illustrative estimate: PHP 12 per kWh. Actual rates vary by provider.
    estimated_bill = (facts["energy"]["monthly_kwh"]
                      * facts["energy"]["rate_per_kwh"])
    budget = facts["energy"]["monthly_budget"]
    if estimated_bill > budget:
        return (f"Estimated bill is PHP {estimated_bill:,.0f}, above the "
                f"PHP {budget:,.0f} budget; review high-use appliances.")
    return None


def rule_check_usage_patterns(facts):
    """Suggest a simple action based on peak-use and lighting habits."""
    habits = facts["usage_patterns"]
    suggestions = []
    if habits["peak_usage_period"] == "afternoon" and not habits["uses_fan"]:
        suggestions.append("Use a fan during afternoon peak-use hours to reduce air-conditioner
demand.")
    if not habits["switches_off_lights_when_away"]:
        suggestions.append("Switch off lights when leaving a room or the home.")
    return " ".join(suggestions) if suggestions else None


def rule_check_aircon_efficiency(facts):
    """Flag long use of a non-inverter air-conditioner."""
    appliance_facts = facts["appliances"]
    if appliance_facts["aircon_hours_daily"] >= 8 and not appliance_facts["has_inverter_aircon"]:
        return "Because the air-conditioner runs for many hours and is not an inverter model, keep
its filter clean and compare efficient replacement options."
    return None


RULES = [
    rule_check_energy_use,
    rule_reduce_aircon_hours,
    rule_use_timer,
    rule_check_budget,
    rule_check_usage_patterns,
    rule_check_aircon_efficiency,
]


def run_inference_engine(facts):
    """Evaluate every rule and return all actions that apply."""
    actions = []
    for rule in RULES:
        action = rule(facts)
        if action:
            actions.append(action)
    return actions


def display_knowledge_base(facts):
    """Show the preset facts that the RBR rules will evaluate."""
    print("RBR facts (preset in KNOWLEDGE_BASE in this code; not entered by the user):")
    for category, category_facts in facts.items():
        print(f"  {category.replace('_', ' ').title()}:")
        for fact_name, value in category_facts.items():
            print(f"    {fact_name.replace('_', ' ')}: {value}")


# ------------------------------------------------------------------------
# Part 3: Case-Based Reasoning (CBR)
# ------------------------------------------------------------------------
```

```
# Problem features are categorical, so similarity is the proportion of
# matching features. Solutions are plain language actions.
CASE_BASE = [
    {
        "case_id": 1,
        "problem": {"usage": "high", "aircon": "long", "home": "empty_day"},
        "solution": "Set an air-conditioner timer and reduce daily cooling hours.",
    },
    {
        "case_id": 2,
        "problem": {"usage": "high", "aircon": "short", "home": "occupied_day"},
        "solution": "Check the refrigerator seal and switch off unused appliances.",
    },
    {
        "case_id": 3,
        "problem": {"usage": "normal", "aircon": "long", "home": "occupied_day"},
        "solution": "Raise the air-conditioner setting slightly and use a fan.",
    },
]

FEATURES = ("usage", "aircon", "home")
ALLOWED_VALUES = {
    "usage": {"high", "normal"},
    "aircon": {"long", "short"},
    "home": {"empty_day", "occupied_day"},
}


def ask_choice(prompt, choices):
    """Require one listed answer; blank or invalid input repeats the prompt."""
    while True:
        answer = input(f"{prompt} ({'/'.join(choices)}): ").strip().lower()
        if answer in choices:
            return answer
        print("Please type one of the listed choices. This question is required.")


def ask_new_problem():
    """Collect CBR problem features from the user."""
    print("Describe the household energy problem using these features.")
    usage = ask_choice("Monthly use", ("high", "normal"))
    aircon = ask_choice("Air-conditioner use", ("long", "short"))
    home = ask_choice("Daytime occupancy", ("empty_day", "occupied_day"))
    return {"usage": usage, "aircon": aircon, "home": home}


def similarity(problem, old_problem):
    """Return the fraction of comparable features that match (0.0 to 1.0)."""
    comparable = [key for key in FEATURES
                  if key in problem and key in old_problem]
    if not comparable:
        return 0.0
    matches = sum(problem[key] == old_problem[key] for key in comparable)
    return matches / len(comparable)


def retrieve_most_similar(problem, cases):
    """Retrieve the best case; ties keep the earliest stored case."""
    if not cases:
        return None, 0.0
    scored_cases = [(similarity(problem, case["problem"]), case) for case in cases]
    best_score = max(score for score, _ in scored_cases)
    best_case = next(case for score, case in scored_cases if score == best_score)
    return best_case, best_score
```

```
def revise_solution(proposed_solution):
    """Reuse a proposal, then let the user accept or replace it."""
    print(f"Proposed solution: {proposed_solution}")
    choice = ask_choice("Accept proposed solution?", ("y", "n"))
    if choice == "y":
        return proposed_solution
    while True:
        revised = input("Enter the revised solution: ").strip()
        if revised:
            return revised
        print("The revised solution cannot be empty. Please try again.")


def run_case_based_reasoning(cases):
    """Run Retrieve, Reuse, Revise, and Retain for one new problem."""
    problem = ask_new_problem()
    print(f"\nNew problem: {problem}")

    # Retrieve: find the closest previous problem.
    retrieved_case, score = retrieve_most_similar(problem, cases)
    if retrieved_case is None or score == 0:
        proposed = "Start with a basic energy audit and identify the largest loads."
        print("Retrieved case: No matching case; using a general starting suggestion.")
    else:
        proposed = retrieved_case["solution"]
        print(f"Retrieved case: Case {retrieved_case['case_id']} - {retrieved_case['problem']}")
    print(f"Similarity: {score:.0%}")

    # Reuse: propose the retrieved solution (or a fallback if no match exists).
    print("Reuse: adapting the retrieved solution to the new problem.")

    # Revise: accept or replace the proposed solution.
    final_solution = revise_solution(proposed)
    print(f"Final solution: {final_solution}")

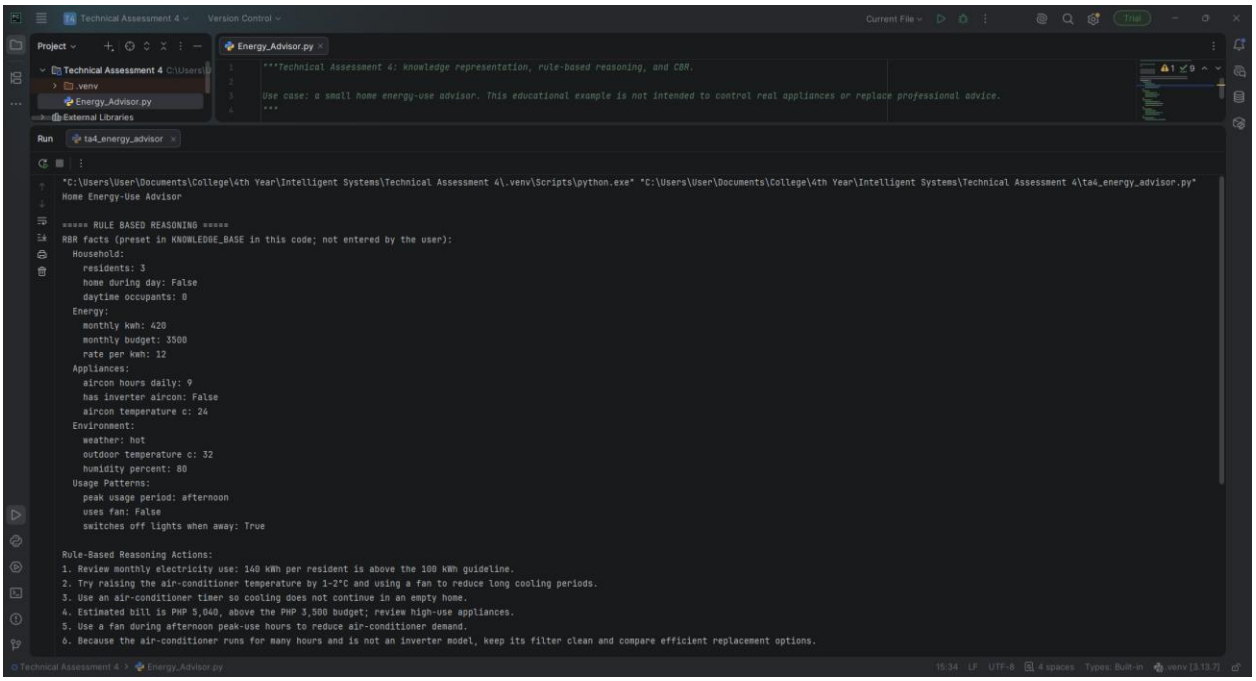
    # Retain: remember the new problem and its final solution for later cases.
    new_case = {
        "case_id": max((case["case_id"] for case in cases), default=0) + 1,
        "problem": problem.copy(),
        "solution": final_solution,
    }
    cases.append(new_case)
    print(f"Retained case: Case {new_case['case_id']} added to the case base.")


def main():
    print("Home Energy-Use Advisor")
    print("\n===== RULE BASED REASONING =====")
    display_knowledge_base(KNOWLEDGE_BASE)
    print("\nRule-Based Reasoning Actions:")
    actions = run_inference_engine(KNOWLEDGE_BASE)
    if actions:
        for number, action in enumerate(actions, start=1):
            print(f"{number}. {action}")
    else:
        print("No rule conditions were met; no actions are recommended.")

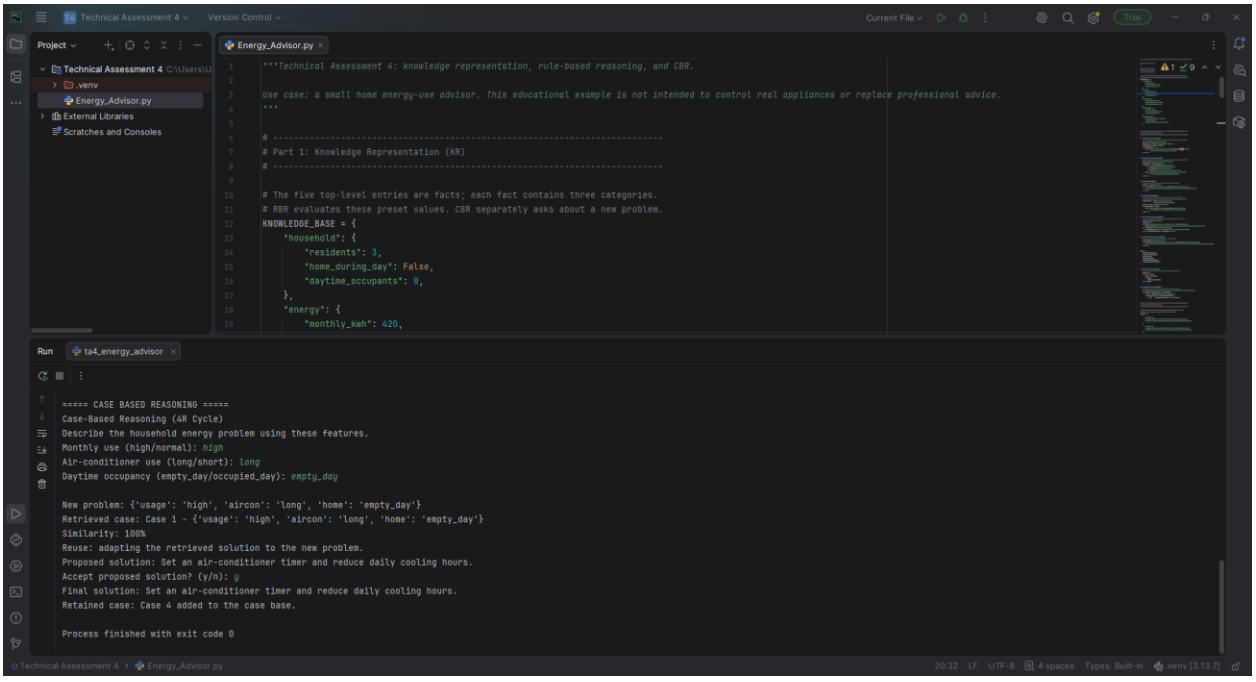
    print("\n===== CASE BASED REASONING =====")
    print("Case-Based Reasoning (4R Cycle)")
    run_case_based_reasoning(CASE_BASE)


if __name__ == "__main__":
    main()
```

Output Screenshot (RBR)



Output Screenshot (CBR)



Reflection:

Rule-Based Reasoning is easier to implement because it checks fixed facts against simple IF–THEN conditions. Case-Based Reasoning required a case base, similarity matching, and the Retrieve, Reuse, Revise, and Retain steps. CBR is more flexible for less structured problems because it can use similar past cases to suggest a solution. RBR works better when data changes continuously and the rules can be updated to reflect the latest information. CBR can also adapt by retaining new cases, though that case base needs maintenance. Real-world RBR examples include eligibility checks and equipment monitoring, while CBR is used in help-desk troubleshooting, medical decision support, and customer support.